第 2 章 搜索

1. 什么是搜索?

搜索，是一种枚举，通过穷举所有的情况来找到最优解，或者统计合法解的个数。因此，搜索有时候也叫作暴搜。

搜索一般分为**深度优先搜索(DFS)与宽度优先搜索(BFS)**。

2. 深度优先遍历 vs 深度优先搜索，宽度优先遍历 vs 宽度优先搜索

遍历是形式，搜索是目的。

不过，在一般情况下，我们不会去纠结概念的差异，两者可以等同。

3. 回溯与剪枝

- 回溯：当在搜索的过程中，遇到走不通或者走到底的情况时，就回头。
- 剪枝：剪掉在搜索过程中，剪掉重复出现或者不是最优解的分支。

1. 深度优先搜索 - DFS

1.1 递归型枚举与回溯剪枝初识

1. 画决策树；
2. 根据决策树写递归。

1.1.1 枚举子集

> **题目来源：** 洛谷
>
> **题目链接：** B3622 枚举子集（递归实现指数型枚举）
>
> **难度系数：** ★

【题目描述】

> 今有 n 位同学，可以从中选出任意名同学参加合唱。
>
> 请输出所有可能的选择方案。

【输入描述】

> 仅一行，一个正整数 n 。

【输出描述】

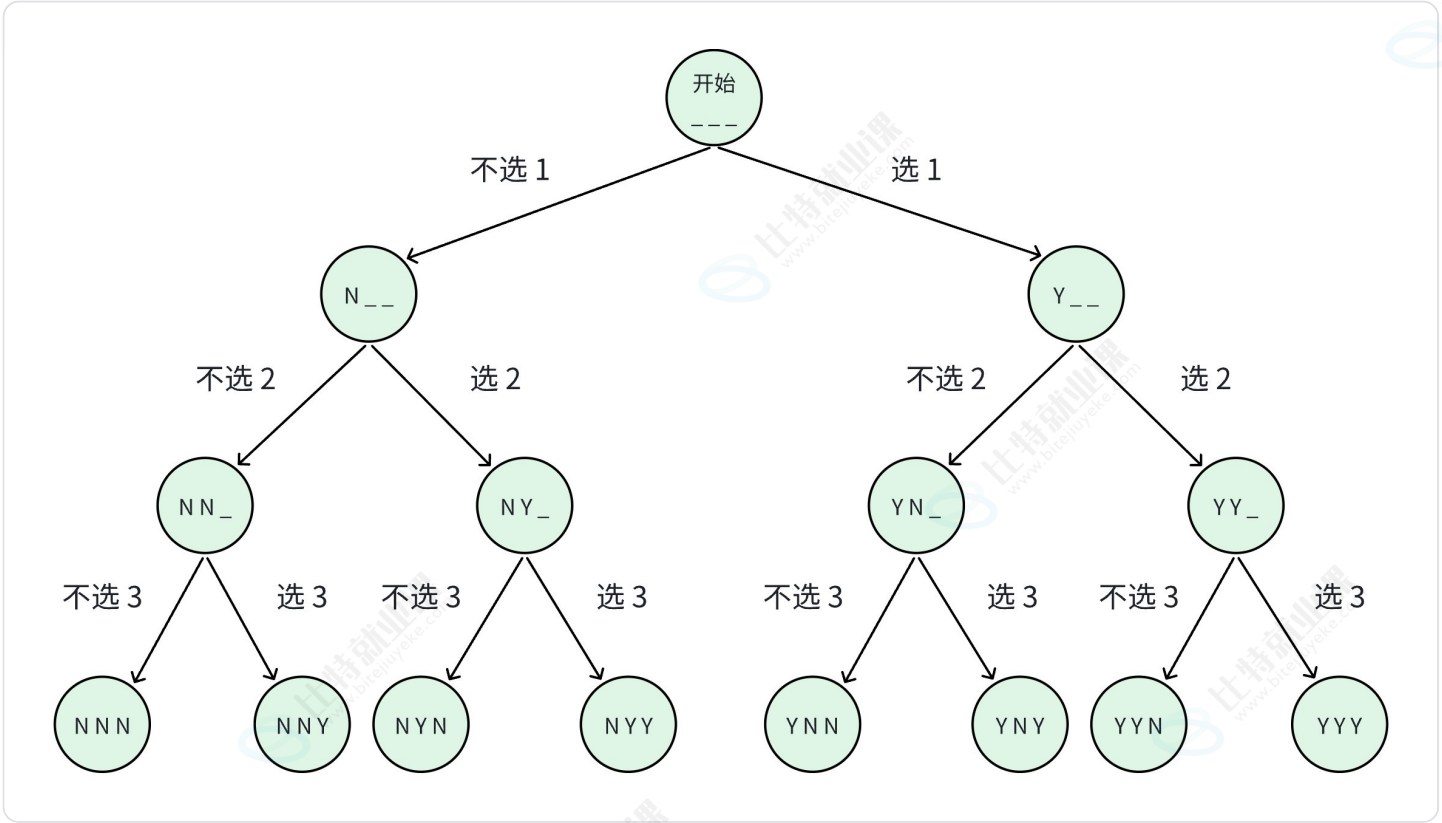
> 若干行，每行表示一个选择方案。
>
> 每一种选择方案用一个字符串表示，其中第 i 位为 `Y` 则表示第 i 名同学参加合唱；为 `N` 则表示不参加。
>
> 需要以字典序输出答案。
>
> 对于 100% 的数据，保证 $1 \leq n \leq 10$ 。

【示例一】

> 输入：
>
> ```
> 3
> ```
>
> 输出：
>
> ```
> NNN
> NNY
> NYN
> NYY
> YNN
> YNY
> YYN
> YYY
> ```

【解法】

设一共有 3 个数，分别是 $[1,2,3]$ 。「从前往后」考虑每一个数，针对当前这个数「选」或者「不选」，我们可以画出如下「决策树」：



设计递归函数：

- 重复子问题：针对某一位，「选」或者「不选」这个数。因为最终结果要按照「字典序」输出，我们可以「先考虑不选」，然后「再考虑选」；
- 实现方式参考代码和注释，结合「决策树」一起看会很清晰。

【参考代码】

```cpp
#include <iostream>

using namespace std;

int n;
string path; // 记录递归过程中，每一步的决策

void dfs(int pos)
{
    if(pos > n)
    {
        // path 就存着前 n 个人的决策
        cout << path << endl;
        return;
    }

```

```cpp
    // 不选
    path += 'N';
    dfs(pos + 1);
    path.pop_back(); // 回溯，清空现场

    // 选
    path += 'Y';
    dfs(pos + 1);
    path.pop_back(); // 清空现场
}

int main()
{
    cin >> n;

    dfs(1);

    return 0;
}
```

1.1.2 组合型枚举

题目来源： 洛谷

题目链接： P10448 组合型枚举

难度系数： ★

【题目描述】

从 $1 \sim n$ 这 n 个整数中随机选出 m 个，输出所有可能的选择方案。

【输入描述】

两个整数 n, m，在同一行用空格隔开。

【输出描述】

按照从小到大的顺序输出所有方案，每行 1 个。

首先，同一行内的数升序排列，相邻两个数用一个空格隔开。

其次，对于两个不同的行，对应下标的数一一比较，字典序较小的排在前面（例如 `1 3 5 7` 排在 `1 3 6 8` 前面）。

对于所有测试数据满足 $0 \leq m \leq n, n + (n - m) \leq 25$。

【示例一】

输入：

5 3

输出：

1 2 3

1 2 4

1 2 5

1 3 4

1 3 5

1 4 5

2 3 4

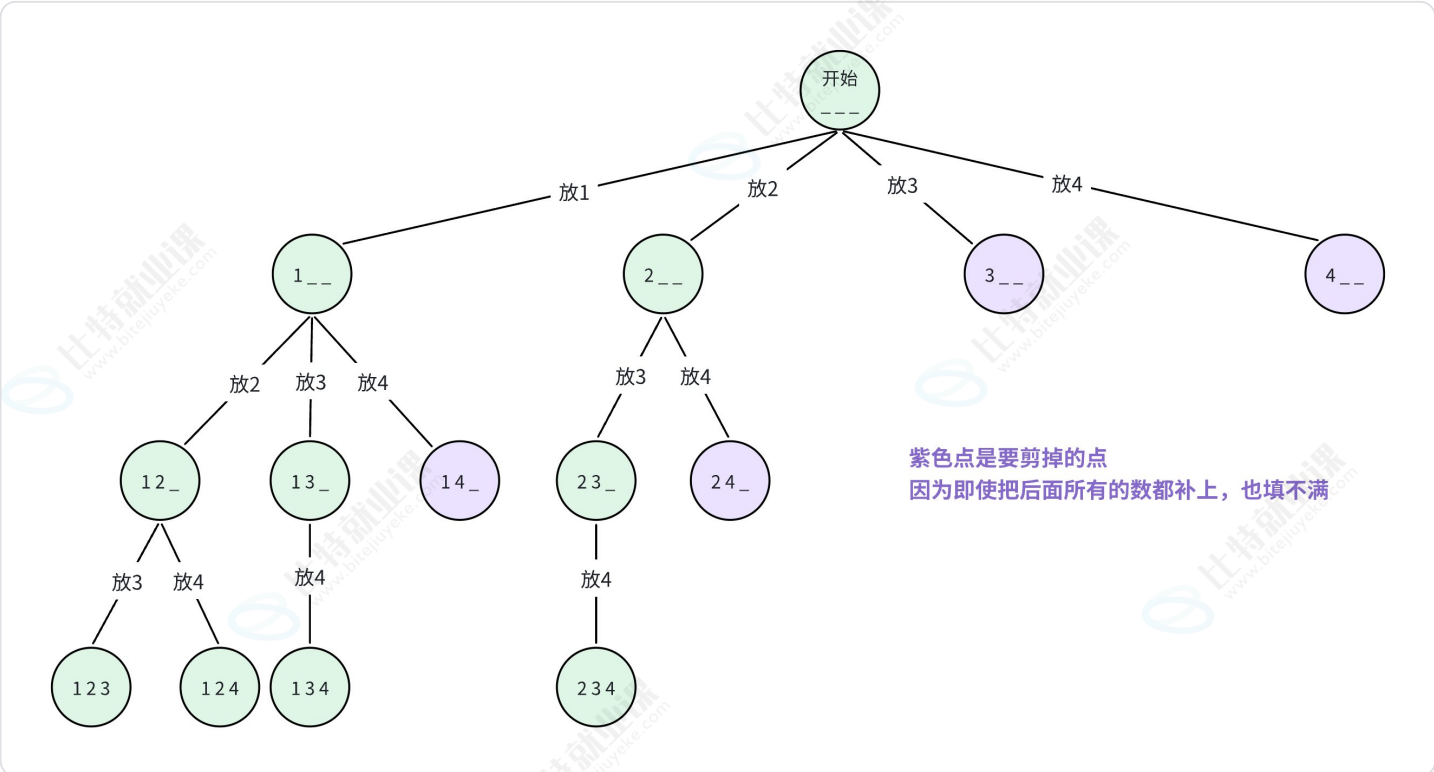
2 3 5

2 4 5

3 4 5

【解法】

设 $n = 4, m = 3$，「从前往后」考虑 3 个位置应该选哪个数，我们可以画出如下决策树：



设计递归函数：

- 重复子问题：当前这一位，应该放哪个数上去。因为这是一个「组合」问题，不涉及排列，所以我们当前位置开始放的数，应该是「上次决策的数的下一位」。

- 实现方式参考代码和注释，结合「决策树」一起看会很清晰。

【参考代码】

```cpp
#include <iostream>
#include <vector>

using namespace std;

int n, m;
vector<int> path;
// path.size();

void dfs(int begin)
{
    if(path.size() == m)
    {
        for(auto x : path) cout << x << " ";
        cout << endl;
        return;
    }

    for(int i = begin; i <= n; i++)
    {
        path.push_back(i);
        dfs(i + 1);
        path.pop_back(); // 清空现场
    }
}

int main()
{
    cin >> n >> m;

    dfs(1);

    return 0;
}
```

1.1.3 枚举排列

> **题目来源： 洛谷**

> 题目链接： B3623 枚举排列（递归实现排列型枚举）
>
> 难度系数： ★

【题目描述】

> 今有 n 名学生，要从中选出 k 人排成一列拍照。
>
> 请按字典序输出所有可能的排列方式。

【输入描述】

> 仅一行，两个正整数 n, k。
>
> 对于 100% 的数据， $1 \leq k \leq n \leq 10$。

【输出描述】

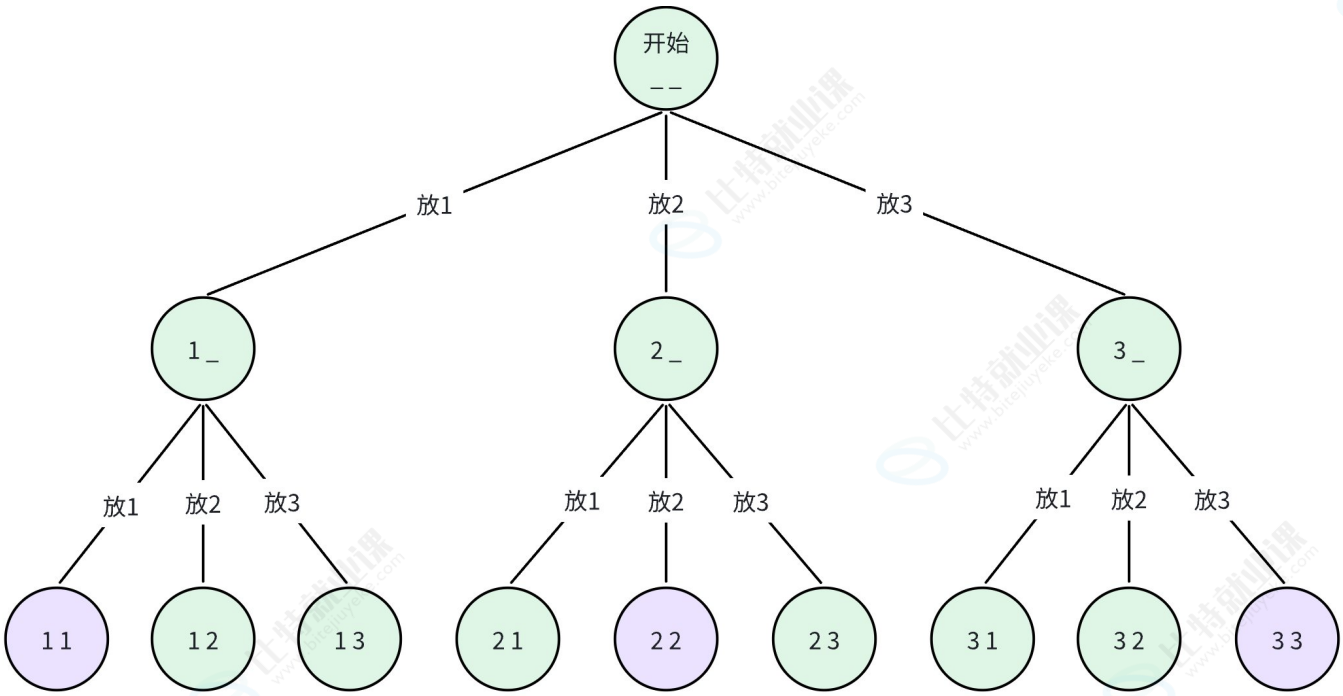
> 若干行，每行 k 个正整数，表示一种可能的队伍顺序。

【示例一】

> 输入：
>
> ```
> 3 2
> ```
>
> 输出：
>
> ```
> 1 2
> 1 3
> 2 1
> 2 3
> 3 1
> 3 2
> ```

【解法】

设 $n=3, k=2$，一共要选出两个数，可以依次「考虑要选出来的数」是谁，画出如下决策树：



紫色部分要剪掉，因为我们不能选择重复的数

设计递归函数：

- 重复子问题：考虑这一位要放上什么数。因为是「排列」问题，所以我们直接从 1 开始枚举要放的数。
- 剪枝：在这一条路径中，我们「不能选择之前已经选择过的数」。
- 实现方式参考代码和注释，结合「决策树」一起看会很清晰。

【参考代码】

```cpp
#include <iostream>
#include <vector>

using namespace std;

const int N = 15;

int n, k;
vector<int> path;
bool st[N]; // 标记一下哪些数已经选过了

void dfs()
{
```

```cpp
    if(path.size() == k)
    {
        for(auto x : path) cout << x << " ";
        cout << endl;
        return;
    }

    for(int i = 1; i <= n; i++)
    {
        if(st[i]) continue;
        path.push_back(i);
        st[i] = true;
        dfs();
        // 恢复现场
        path.pop_back();
        st[i] = false;
    }
}

int main()
{
    cin >> n >> k;

    dfs();

    return 0;
}
```

1.1.4 全排列问题

> **题目来源：** 洛谷
>
> **题目链接：** P1706 全排列问题
>
> **难度系数：** ★

【题目描述】

> 按照字典序输出自然数 1 到 n 所有不重复的排列，即 n 的全排列，要求所产生的任一数字序列中不允许出现重复的数字。

【输入描述】

> 一个整数 $n(1 \le n \le 9)$。

【输出描述】

> 由 $1 \sim n$ 组成的所有不重复的数字序列，每行一个序列。

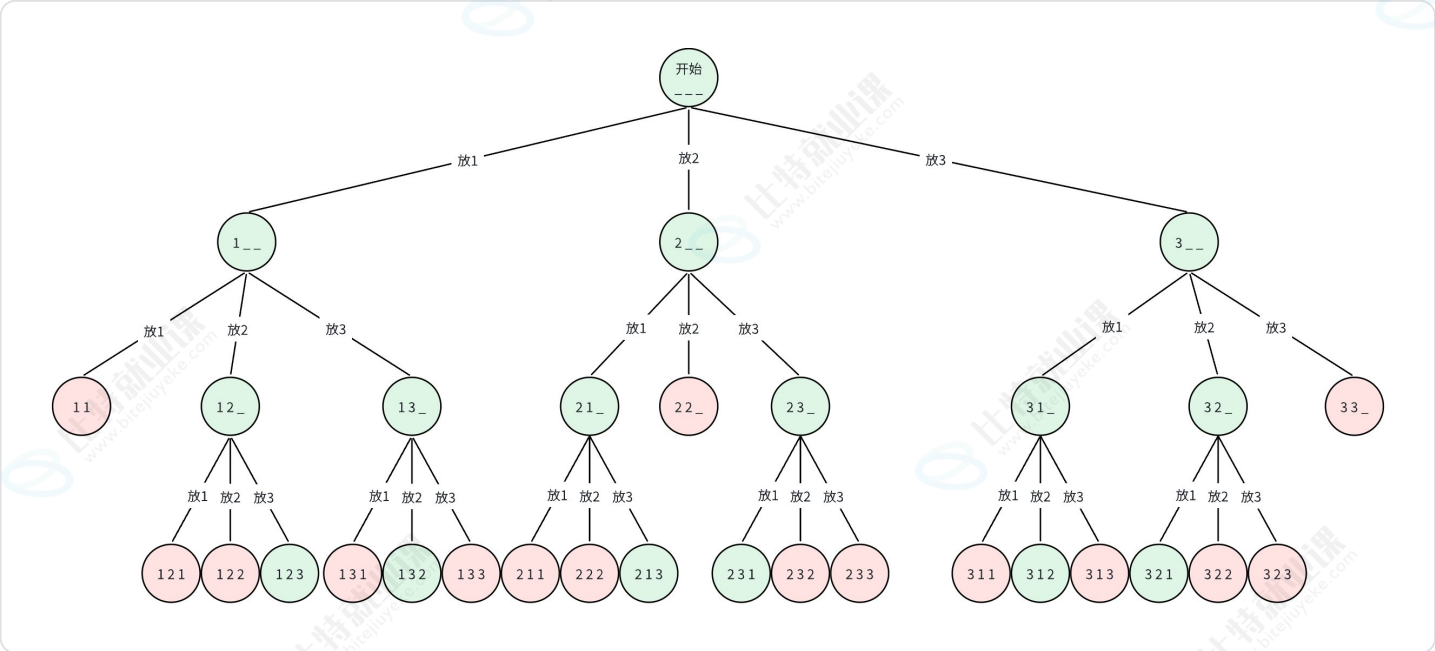
> 每个数字保留 5 个场宽。

【示例一】

> 输入：
>
> 3
>
> 输出：
>
> 1 2 3
>
> 1 3 2
>
> 2 1 3
>
> 2 3 1
>
> 3 1 2
>
> 3 2 1

【解法】

跟上一道题的决策一样，我们可以枚举每一位应该放上什么数，只不过少了 k 的限制。剪枝的策略还是一样的，那就是在路径中，「不能选择之前已经选过的数」。



关于「全排列问题」，我们也可以调用 STL 里面的「$next_permutation$ 函数」。

【参考代码】

```
#include <iostream>
#include <vector>
```

```cpp

using namespace std;

const int N = 15;

int n;
bool st[N];
vector<int> path;

void dfs()
{
    if(path.size() == n)
    {
        for(auto x : path)
        {
            printf("%5d", x);
        }
        cout << endl;
        return;
    }

    for(int i = 1; i <= n; i++)
    {
        if(st[i]) continue;

        path.push_back(i);
        st[i] = true;
        dfs();
        // 恢复现场
        path.pop_back();
        st[i] = false;
    }
}

int main()
{
    cin >> n;

    dfs();

    return 0;
}
```

1.2 DFS

1.2.1 选数

> **题目来源：** 洛谷
>
> **题目链接：** P1036 [NOIP2002 普及组] 选数
>
> **难度系数：** ★

【题目描述】

> 已知 n 个整数 $x_1, x_2, x_3, ..., x_n$，以及 1 个整数 $k(k < n)$。从 n 个整数中任选 k 个整数相加，可分别得到一系列的和。例如当 $n = 4, k = 3$，4 个整数分别为 $3, 7, 12, 19$ 时，可得全部的组合与它们的和为：
>
> $3 + 7 + 12 = 22$
>
> $3 + 7 + 19 = 29$
>
> $7 + 12 + 19 = 38$
>
> $3 + 12 + 19 = 34$
>
> 现在，要求你计算出和为素数共有多少种。
>
> 例如上例，只有一种的和为素数：$3 + 7 + 19 = 29$

【输入描述】

> 第一行两个空格隔开的整数 $n, k(1 \le n \le 20, k < n)$。
>
> 第二行 n 个整数，分别为 $x_1, x_2, ..., x_n(1 \le x_i \le 5 \times 10^6)$。

【输出描述】

> 输出一个整数，表示种类数。

【示例一】

> 输入：
>
> ```
> 4 3
> 3 7 12 19
> ```
>
> 输出：
>
> ```
> 1
> ```

【解法】

「组合」型枚举，路径里面记录选择数的「总和」。在选出 k 个数之后，判断「是否是质数」。

【参考代码】

```
1  #include <iostream>
2
```

```cpp
using namespace std;

const int N = 25;

int n, k;
int a[N];

int ret;
int path; // 记录路径中所选择的数的和

bool isprime(int x)
{
    if(x <= 1) return false;
    // 试除法
    for(int i = 2; i <= x / i; i++)
    {
        if(x % i == 0) return false;
    }
    return true;
}

void dfs(int pos, int begin)
{
    if(pos > k)
    {
        if(isprime(path)) ret++;
        return;
    }

    for(int i = begin; i <= n; i++)
    {
        path += a[i];
        dfs(pos + 1, i + 1);
        path -= a[i]; // 恢复现场
    }
}

int main()
{
    cin >> n >> k;
    for(int i = 1; i <= n; i++) cin >> a[i];

    dfs(1, 1);

    cout << ret << endl;

    return 0;
```

```
50  }
```

1.2.2 飞机降落

题目来源： 洛谷

题目链接： P9241 [蓝桥杯 2023 省 B] 飞机降落

难度系数： ★★

【题目描述】

N 架飞机准备降落到某个只有一条跑道的机场。其中第 i 架飞机在 T_i 时刻到达机场上空，到达时它的剩余油料还可以继续盘旋 D_i 个单位时间，即它最早可以于 T_i 时刻开始降落，最晚可以于 $T_i + D_i$ 时刻开始降落。降落过程需要 L_i 个单位时间。

一架飞机降落完毕时，另一架飞机可以立即在同一时刻开始降落，但是不能在前一架飞机完成降落前开始降落。

请你判断 N 架飞机是否可以全部安全降落。

【输入描述】

输入包含多组数据。

第一行包含一个整数 T，代表测试数据的组数。

对于每组数据，第一行包含一个整数 N。

以下 N 行，每行包含三个整数：T_i, D_i, L_i。

对于 100% 的数据，$1 \leq T \leq 10$，$1 \leq N \leq 10$，$0 \leq T_i, D_i, Li \leq 10^5$。

【输出描述】

对于每组数据，输出 YES 或者 NO，代表是否可以全部安全降落。

【示例一】

输入：

```
2
3
0 100 10
10 10 10
0 2 20
3
```

> 0 10 20
>
> 10 10 20
>
> 20 10 20
>
> 输出：
>
> YES
>
> NO

【解法】

枚举所有飞机的「全排列」，判断是否存在一种排列，使的全部的飞机都能安全降落。

剪枝：

- 当前路径里面只能选没有选过的飞机；
- 如果这架飞机不能正常降落，剪掉；
- 如果已经找到一种安全降落的方式，停止枚举，可以通过「递归的返回值」判断是否搜索成功。

【参考代码】

```cpp
#include <iostream>
#include <cstring>

using namespace std;

const int N = 15;

int n;
int t[N], d[N], l[N];

bool st[N]; // 标记路径中哪些飞机已经摆放过

bool dfs(int pos, int end)
{
    if(pos > n)
    {
        return true;
    }

    for(int i = 1; i <= n; i++)
    {
        if(st[i] == true) continue; // 剪枝
        if(end > t[i] + d[i]) continue; // 剪枝
        int newend = max(t[i], end) + l[i];
```

```
            st[i] = true;
            if(dfs(pos + 1, newend)) return true;
            st[i] = false; // 回复现场
        }

        return false;
    }

    int main()
    {
        int T; cin >> T;

        while(T--) // 多组测试数据的时候，一定要注意清空数据
        {
            memset(st, 0, sizeof st);
            cin >> n;
            for(int i = 1; i <= n; i++) cin >> t[i] >> d[i] >> l[i];

            if(dfs(1, 0)) cout << "YES" << endl;
            else cout << "NO" << endl;
        }

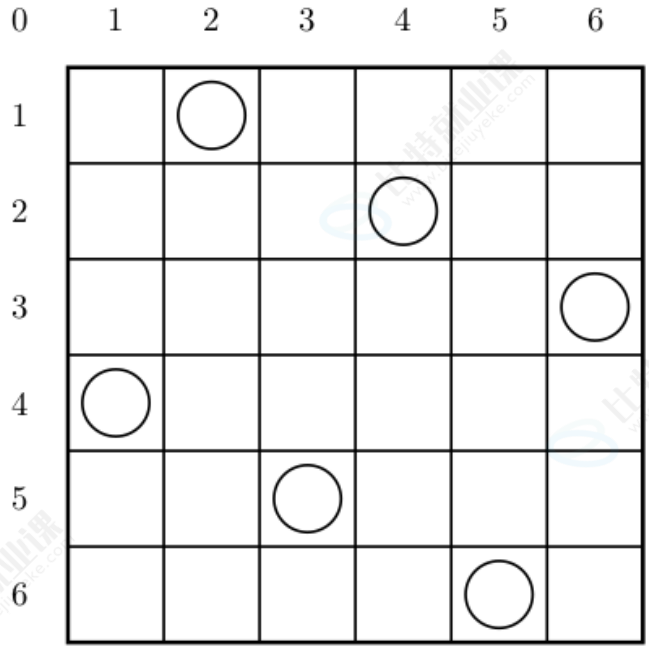

        return 0;
    }
```

1.2.3 八皇后

> **题目来源：** 洛谷
>
> **题目链接：** P1219 [USACO1.5] 八皇后 Checker Challenge
>
> **难度系数：** ★★

【题目描述】

> 一个如下的 6×6 的跳棋棋盘，有六个棋子被放置在棋盘上，使得每行、每列有且只有一个，每条对角线（包括两条主对角线的所有平行线）上至多有一个棋子。



洛谷

上面的布局可以用序列 $2\,4\,6\,1\,3\,5$ 来描述，第 i 个数字表示在第 i 行的相应位置有一个棋子，如下：

行号 $1\,2\,3\,4\,5\,6$

列号 $2\,4\,6\,1\,3\,5$

这只是棋子放置的一个解。请编一个程序找出所有棋子放置的解。

并把它们以上面的序列方法输出，解按字典顺序排列。

请输出前 3 个解。最后一行是解的总个数。

【输入描述】

一行一个正整数 n，表示棋盘是 $n \times n$ 大小的。

对于 100% 的数据，$6 \leq n \leq 13$。

【输出描述】

前三行为前三个解，每个解的两个数字之间用一个空格隔开。第四行只有一个数字，表示解的总数。

【示例一】

输入：

```
6
```

输出：

```
2 4 6 1 3 5
3 6 2 5 1 4
```

```
4 1 5 2 6 3
4
```

【解法】

枚举策略：

- 「一行一行」的放皇后：从第一行开始，尝试在每一列上放皇后；
- 如果当前列放上皇后之后「没有冲突」，就标记一下这个「放法」，记录一下当前行的决策，然后「递归」考虑下一行；
- 等到「所有行」都放置完毕之后，输出本次枚举的决策。

枚举策略应该是比较容易想到的，这道题的难点在于如何判断「在这一列放上这个皇后之后，是否冲突」。当我们一行一行放的时候，「行是不会产生冲突的」。产生冲突的只有「列」，「主对角线」，以及「副对角线」。我们可以用三个数组分别标记：

- $col[i] = true$，表示第 i 行放置了一个皇后；
- $dig1[j - i + n] = true$，表示 $y = x + (j - i)$ 这条「主对角线」上放置了一个皇后；
- $dig2[j + i] = true$，表示 $y = -x + (j + i)$ 这条「副对角线」上放置了一个皇后。

【参考代码】

```cpp
#include <iostream>
#include <vector>

using namespace std;

const int N = 15;

int n;
bool col[N], st1[N * 2], st2[N * 2];

int ret;
vector<int> path;

void dfs(int x)
{
    if(x > n)
    {
        ret++;
        if(ret <= 3)
        {
```

```
            for(auto x : path) cout << x << " ";
            cout << endl;
        }
        return;
    }

    for(int y = 1; y <= n; y++)
    {
        // 判断能不能摆在这一列
        if(col[y] || st1[y - x + n] || st2[y + x]) continue; // 剪枝
        col[y] = st1[y - x + n] = st2[y + x] = true;
        path.push_back(y);
        dfs(x + 1);
        col[y] = st1[y - x + n] = st2[y + x] = false;
        path.pop_back();
    }
}

int main()
{
    cin >> n;

    dfs(1);

    cout << ret << endl;

    return 0;
}
```

1.2.4 数独

> **题目来源：** 洛谷
>
> **题目链接：** P1784 数独
>
> **难度系数：** ★★

【题目描述】

> 数独是根据 9×9 盘面上的已知数字，推理出所有剩余空格的数字，并满足每一行、每一列、每一个粗线宫内的数字均含 $1 \sim 9$ ，不重复。每一道合格的数独谜题都有且仅有唯一答案，推理方法也以此为基础，任何无解或多解的题目都是不合格的。
>
> 芬兰一位数学家号称设计出全球最难的“数独游戏”，并刊登在报纸上，让大家去挑战。
>
> 这位数学家说，他相信只有“智慧最顶尖”的人才有可能破解这个“数独之谜”。

> 据介绍，目前数独游戏的难度的等级有一到五级，一是入门等级，五则比较难。不过这位数学家说，他所设计的数独游戏难度等级是十一，可以说是所有数独游戏中，难度最高的等级。他还表示，他目前还没遇到解不出来的数独游戏，因此他认为“最具挑战性”的数独游戏并没有出现。

【输入描述】

> 一个未填的数独。

【输出描述】

> 填好的数独。

【示例一】

> 输入：
>
> 800000000
> 003600000
> 070090200
> 050007000
> 000045700
> 000100030
> 001000068
> 008500010
> 090000400
>
> 输出：
>
> 812753649
> 943682175
> 675491283
> 154237896
> 369845721
> 287169534
> 521974368
> 438526917
> 796318452

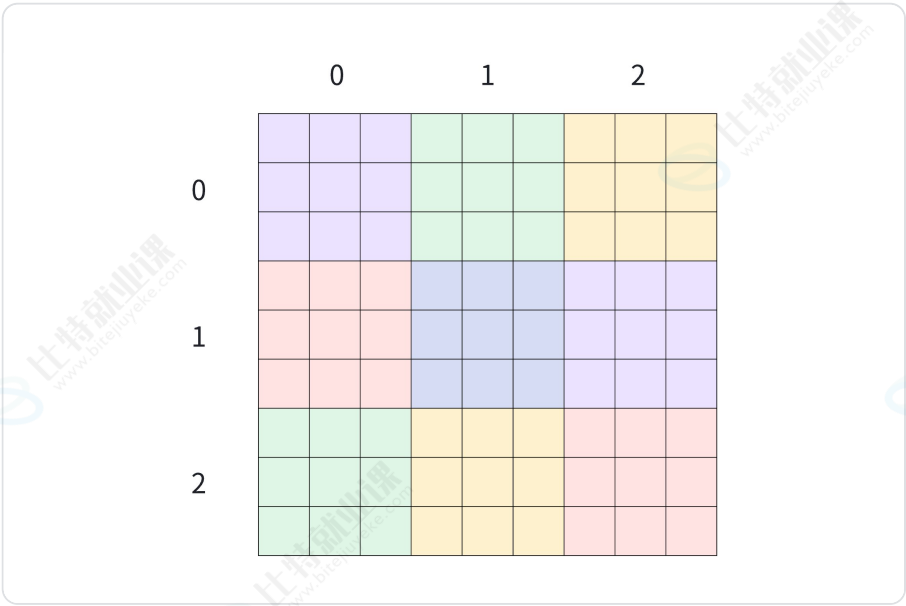
【解法】

枚举策略：

- 「一个格子一个格子」往里面填数。

- 从第一行的第一个格子开始，填上一个「没有冲突」的数，然后「递归」到下一个格子；
- 当某一行填满之后，递归到「下一行的起始位置」继续填数。

可以创建三个数组，用来帮助判断填上某一个数之后，是否会发生冲突。对于 3×3 方格，我们可以给每一个格子编上号，快读定位，如下图：



- $row[i][num] = true$ 表示：第 i 行已经放上了 num 这个数；
- $col[j][num] = true$ 表示：第 j 列已经放上了 num 这个数；
- $st[i/3][j/3][num] = true$ 表示： $[i/3, j/3]$ 的 3×3 方格里，已经放上了 num 这个数。

【参考代码】

```cpp
#include <iostream>

using namespace std;

const int N = 10;

int n = 9;
int a[N][N];
bool row[N][N], col[N][N], st[N][N][N];

bool dfs(int i, int j)
{
    if(j == n)
    {
        // 当这一行填满之后
        i++;
```

```cpp
        j = 0;
    }

    if(i == n) return true; // 找到一种合法的情况，就停止递归
    if(a[i][j]) return dfs(i, j + 1);

    for(int x = 1; x <= 9; x++)
    {
        if(row[i][x] || col[j][x] || st[i / 3][j / 3][x]) continue; // 剪枝

        row[i][x] = col[j][x] = st[i / 3][j / 3][x] = true;
        a[i][j] = x;

        if(dfs(i, j + 1)) return true;

        // 恢复现场
        row[i][x] = col[j][x] = st[i / 3][j / 3][x] = false;
        a[i][j] = 0;
    }

    return false;
}

int main()
{
    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cin >> a[i][j];
            int x = a[i][j];
            if(x)
            {
                // 标记一下
                row[i][x] = col[j][x] = st[i / 3][j / 3][x] = true;
            }
        }
    }

    dfs(0, 0);

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < n; j++)
        {
            cout << a[i][j] << " ";
        }
```

```
64            cout << endl;
65        }
66
67        return 0;
68    }
```

1.3 剪枝与优化

剪枝，形象得看，就是剪掉搜索树的分支，从而减小搜索树的规模，排除掉搜索树中没有必要的分支，优化时间复杂度。

在深度优先遍历中，有几种常见的剪枝方法：

1. **排除等效冗余**

> 如果在搜索过程中，通过某一个节点往下的若干分支中，存在最终结果等效的分支，那么就只需要搜索其中一条分支。

2. **可行性剪枝**

> 如果在搜索过程中，发现有一条分支是无论如何都拿不到最终解，此时就可以放弃这个分支，转而搜索其它的分支。

3. **最优性剪枝**

> 在最优化的问题中，如果在搜索过程中，发现某一个分支已经超过当前已经搜索过的最优解，那么这个分支往后的搜索，必定不会拿到最优解。此时应该停止搜索，转而搜索其它情况。

4. **优化搜索顺序**

> 在有些搜索问题中，搜索顺序是不影响最终结果的，此时搜索顺序的不同会影响搜索树的规模。
>
> 因此，应当先选择一个搜索分支规模较小的搜索顺序，快速拿到一个最优解之后，用最优性剪枝剪掉别的分支。

5. **记忆化搜索**

> 记录每一个状态的搜索结果，当下一次搜索到这个状态时，直接找到之前记录过的搜索结果。
>
> 记忆化搜索，有时也叫动态规划。

1.3.1 数的划分

> **题目来源：** 洛谷
>
> **题目链接：** P1025 [NOIP2001 提高组] 数的划分
>
> **难度系数：** ★★

【题目描述】

> 将整数 n 分成 k 份，且每份不能为空，任意两个方案不相同（不考虑顺序）。
>
> 例如： $n=7,k=3$，下面三种分法被认为是相同的。
>
> $1,1,5$
>
> $1,5,1$
>
> $5,1,1$
>
> 问有多少种不同的分法。

【输入描述】

> n,k $(6<n\leq 200，2\leq k\leq 6)$

【输出描述】

> 1 个整数，即不同的分法。

【示例一】

> 输入：
>
> ```
> 7 3
> ```
>
> 输出：
>
> ```
> 4
> ```

【解法】

搜索策略：

- 将 $[1,n]$ 个数放在 k 个坑里面，使的坑里面的所有数的总和是 n；
- 其中，不同的坑里面的数可能相同；
- 但是 $[1,2]$ 与 $[2,1]$ 是同一种分法，因此，应该是一种组合型枚举。针对每一个坑里面的数应该放谁的时候，应该从上一个坑里面的数开始枚举。

剪枝策略：

- 当我们填了 cnt 个坑时，此时总和是 sum，如果后续坑位全部都填上最小值都会超过 n。说明我们之前填的数太大了，导致后面怎么填都会超过 n，直接剪掉。

注意剪枝位置的不同，而导致搜索树的不同：

- 如果在进入递归之前剪枝，我们不会进入非法的递归函数中；
- 但是如果在进入递归之后剪枝，我们就会多进入很多不合法的递归函数中。

【参考代码】

```cpp
#include <iostream>

using namespace std;

int n, k;
int path, ret;

void dfs(int pos, int begin)
{
    if(pos == k)
    {
        if(path == n) ret++;
        return;
    }

    // 可行性剪枝
    // if(path + begin * (k - pos) > n) return;

    for(int i = begin; i <= n; i++)
    {
        // 可行性剪枝
        if(path + i * (k - pos) > n) return;

        path += i;
        dfs(pos + 1, i);
        path -= i;
    }
}

int main()
{
    cin >> n >> k;

    dfs(0, 1);

    cout << ret << endl;

    return 0;
}
```

1.3.2 小猫爬山

> **题目来源：** 洛谷
>
> **题目链接：** P10483 小猫爬山
>
> **难度系数：** ★★

【题目描述】

> Freda 和 rainbow 饲养了 $N(N \leq 18)$ 只小猫，这天，小猫们要去爬山。经历了千辛万苦，小猫们终于爬上了山顶，但是疲倦的它们再也不想徒步走下山了
>
> Freda 和 rainbow 只好花钱让它们坐索道下山。索道上的缆车最大承重量为 W ，而 N 只小猫的重量分别是 $C_1, C_2, \ldots, C_N$ 。当然，每辆缆车上的小猫的重量之和不能超过 $W(1 \leq C_i \leq W \leq 10^8)$ 。每租用一辆缆车，Freda 和 rainbow 就要付 1 美元，所以他们想知道，最少需要付多少美元才能把这 N 只小猫都运送下山?

【输入描述】

> 第一行包含两个用空格隔开的整数， N 和 W 。 接下来 N 行每行一个整数，其中第 $i+1$ 行的整数表示第 i 只小猫的重量 C_i 。

【输出描述】

> 输出一个整数，最少需要多少美元，也就是最少需要多少辆缆车。

【示例一】

> 输入：
>
> ```
> 5 1996
> 1
> 2
> 1994
> 12
> 29
> ```
>
> 输出：
>
> ```
> 2
> ```

【解法】

搜索策略：依次处理每一只猫，对于每一只猫，我们都有两种处理方式：

- 要么把这只猫放在已经租好的缆车上；
- 要么重新租一个缆车，把这只猫放上去。

剪枝：

- 在搜索过程中，我们用全局变量记录已经搜索出来的最小缆车数量。如果当前搜索过程中，已经用的缆车数量大于全局记录的最小缆车数量，那么这个分支一定不会得到最优解，剪掉。
- 优化枚举顺序一：从大到小安排每一只猫
  - 重量较大的猫能够快速把缆车填满，较快得到一个最小值；
  - 通过这个最小值，能够提前把分支较大的情况提前剪掉。
- 优化枚举策略二：先考虑把小猫放在已有的缆车上，然后考虑重新租一辆车
  - 因为如果反着来，我们会先把缆车较大的情况枚举出来，这样就起不到剪枝的效果了。

【参考代码】

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 20;

int n, w;
int c[N]; // 小猫的信息

int cnt; // 当前用了多少车
int s[N]; // 每一辆车目前的总重

int ret = N; // 最优解

bool cmp(int a, int b)
{
    return a > b;
}

void dfs(int pos)
{
    // 策略二：最优性剪枝
    if(cnt >= ret) return;

    if(pos > n)
    {
        ret = cnt;
        return;
```

```
30      }
31
32      // 策略三：优化搜索顺序
33      // 先安排在已有的车辆上
34      for(int i = 1; i <= cnt; i++)
35      {
36          // 策略一：可行性剪枝
37          if(s[i] + c[pos] > w) continue;
38          s[i] += c[pos];
39          dfs(pos + 1);
40          s[i] -= c[pos]; // 恢复现场
41      }
42
43      // 重开一辆车
44      cnt++;
45      s[cnt] = c[pos];
46      dfs(pos + 1);
47      // 恢复现场
48      s[cnt] = 0;
49      cnt--;
50  }
51
52  int main()
53  {
54      cin >> n >> w;
55      for(int i = 1; i <= n; i++) cin >> c[i];
56
57      // 策略三：优化搜索顺序
58      sort(c + 1, c + 1 + n, cmp);
59
60      dfs(1);
61
62      cout << ret << endl;
63
64      return 0;
65  }
```

1.4 记忆化搜索

记忆化搜索也是一种剪枝策略。

通过一个"备忘录"，记录第一次搜索到的结果，当下一次搜索到这个状态时，直接在"备忘录"里面找结果。

记忆化搜索，有时也叫动态规划。

【案例】

> **题目来源：** 力扣
>
> **题目链接：** 509. 斐波那契数
>
> **难度系数：** ★

【题目描述】

> **斐波那契数** （通常用 `F(n)` 表示）形成的序列称为 **斐波那契数列** 。该数列由 `0` 和 `1` 开始，后面的每一项数字都是前面两项数字的和。也就是：
>
> F(0) = 0，F(1) = 1
>
> F(n) = F(n - 1) + F(n - 2)，其中 n > 1
>
> 给定 `n` ，请计算 `F(n)` 。

【算法原理】

在搜索的过程中，如果发现特别多完全相同的子问题，就可以添加一个备忘录，将搜索的结果放在备忘录中。下一次在搜索到这个状态时，直接在备忘录里面拿值。

【代码实现】

```
1   class Solution
2   {
3       int f[35]; // 搞一个备忘录
4
5   public:
6       int fib(int n)
7       {
8           memset(f, -1, sizeof f); // 先初始化成一定不会存在的值
9
10          return dfs(n);
11      }
12
13      int dfs(int n)
14      {
15          // 搜索之前先往备忘录里面瞅瞅
16          if(f[n] != -1) return f[n];
17
18          if(n == 0 || n == 1) return n;
19
20          // 返回之前，把结果记录在备忘录中
```

```
21            f[n] = dfs(n - 1) + dfs(n - 2);
22            return f[n];
23        }
24    };
```

1.4.1 Function

题目来源： 洛谷

题目链接： P1464 Function

难度系数： ★

【题目描述】

对于一个递归函数 $w(a,b,c)$

- 如果 $a \le 0$ 或 $b \le 0$ 或 $c \le 0$ 就返回值 1 。
- 如果 $a > 20$ 或 $b > 20$ 或 $c > 20$ 就返回 $w(20,20,20)$
- 如果 $a < b$ 并且 $b < c$ 就返回 $w(a,b,c-1)+w(a,b-1,c-1)-w(a,b-1,c)$
- 其它的情况就返回
  $w(a-1,b,c)+w(a-1,b-1,c)+w(a-1,b,c-1)-w(a-1,b-1,c-1)$

这是个简单的递归函数，但实现起来可能会有些问题。当 a,b,c 均为 15 时，调用的次数将非常的多。你要想个办法才行。

注意：例如 $w(30,-1,0)$ 又满足条件 1 又满足条件 2 ，请按照最上面的条件来算，答案为 1 。

【输入描述】

会有若干行。

并以 $-1,-1,-1$ 结束。

保证输入的数在 $[-9223372036854775808, 9223372036854775807]$ 之间，并且是整数。

保证不包括 $-1,-1,-1$ 的输入行数 T 满足 $1 \le T \le 10^5$ 。

【输出描述】

输出若干行，每一行格式：

`w(a, b, c) = ans`

注意空格。

【示例一】

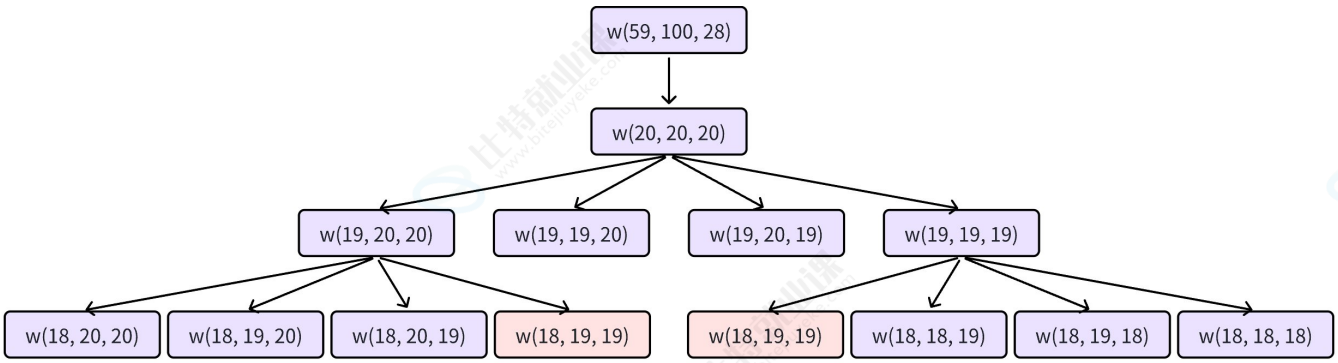
输入：

> 1 1 1
>
> 2 2 2
>
> -1 -1 -1
>
> 输出：
>
> w(1, 1, 1) = 2
>
> w(2, 2, 2) = 4

【解法】

题目叙述的非常清楚，我们仅需按照「题目的要求」把「递归函数」写出来即可。但是，如果不做其余处理的话，结果会「超时」。因为我们递归的「深度」和「广度」都非常大。

通过把「递归展开图」画出来，我们发现，在递归过程中会遇到大量「一模一样」的问题，如下图（因为递归展开过于庞大，这里只画出了一部分）：



因此，可以在递归的过程中，把每次算出来的结果存在一张「备忘录」里面。等到下次递归进入「一模一样」的问题之后，就「不用傻乎乎的展开计算」，而是在「备忘录里面直接把结果拿出来」，起到大量剪枝的效果。

【参考代码】

```
#include <iostream>

using namespace std;

typedef long long LL;

const int N = 25;

LL a, b, c;
LL f[N][N][N]; // 备忘录

```

```cpp
LL dfs(LL a, LL b, LL c)
{
    if(a <= 0 || b <= 0 || c <= 0) return 1;
    if(a > 20 || b > 20 || c > 20) return dfs(20, 20, 20);

    if(f[a][b][c]) return f[a][b][c];

    if(a < b && b < c) return f[a][b][c] = dfs(a, b, c - 1) + dfs(a, b - 1, c - 1) - dfs(a, b - 1, c);
    else return f[a][b][c] = dfs(a - 1, b, c) + dfs(a - 1, b - 1, c) + dfs(a - 1, b, c - 1) - dfs(a - 1, b - 1, c - 1);
}

int main()
{
    while(cin >> a >> b >> c)
    {
        // 多组测试数据：不需要清空
        if(a == -1 && b == -1 && c == -1) break;

        printf("w(%lld, %lld, %lld) = %lld\n", a, b, c, dfs(a, b, c));
    }

    return 0;
}
```

1.4.2 天下第一

题目来源： 洛谷

题目链接： P5635【CSGRound1】天下第一

难度系数： ★★

【题目描述】

游戏是这样的：

给定两个数 x, y，与一个模数 p。

cbw 拥有数 x，zhouwc 拥有数 y。

第一个回合：$x \leftarrow (x + y) \bmod p$

第二个回合：$y \leftarrow (x + y) \bmod p$

第三个回合：$x \leftarrow (x + y) \bmod p$

第四个回合：$y \leftarrow (x + y) \bmod p$